Anabela Petrushevikj D208 PA Part II Logistic RegressionResearch question: "How do factors such as being overweight, having high blood pressure, initial hospital days, doctor’s visits, and the method of admission influence the likelihood of a patient being readmitted to the hospital?"

In [1]:
```
#Importing libraries
import pandas as pd
import numpy as np
import seaborn as sns
import sklearn
import matplotlib.pyplot as plt
from scipy.stats import zscore
import statsmodels.api as sm
from statsmodels.formula.api import ols
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.metrics import confusion_matrix, accuracy_score, ConfusionMatrixDisplay
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.graphics.mosaicplot import mosaic
from scipy import stats
```

In [2]:
```
#Loading the dataset
df = pd.read_csv('medical_clean_D208_II.csv')
```

In [3]:
```
#Information about the dataset
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 50 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   CaseOrder           10000 non-null  int64
 1   Customer_id         10000 non-null  object
 2   Interaction         10000 non-null  object
 3   UID                 10000 non-null  object
 4   City                10000 non-null  object
 5   State               10000 non-null  object
 6   County              10000 non-null  object
 7   Zip                 10000 non-null  int64
 8   Lat                 10000 non-null  float64
 9   Lng                 10000 non-null  float64
 10  Population          10000 non-null  int64
 11  Area                10000 non-null  object
 12  TimeZone            10000 non-null  object
 13  Job                 10000 non-null  object
 14  Children            10000 non-null  int64
 15  Age                 10000 non-null  int64
 16  Income              10000 non-null  float64
 17  Marital             10000 non-null  object
 18  Gender              10000 non-null  object
 19  ReAdmis             10000 non-null  object
 20  VitD_levels         10000 non-null  float64
 21  Doc_visits          10000 non-null  int64
 22  Full_meals_eaten    10000 non-null  int64
 23  vitD_supp           10000 non-null  int64
 24  Soft_drink          10000 non-null  object
 25  Initial_admin       10000 non-null  object
 26  HighBlood           10000 non-null  object
 27  Stroke              10000 non-null  object
 28  Complication_risk   10000 non-null  object
 29  Overweight          10000 non-null  object
 30  Arthritis           10000 non-null  object
 31  Diabetes            10000 non-null  object
 32  Hyperlipidemia      10000 non-null  object
 33  BackPain            10000 non-null  object
 34  Anxiety             10000 non-null  object
 35  Allergic_rhinitis   10000 non-null  object
 36  Reflux_esophagitis  10000 non-null  object
 37  Asthma              10000 non-null  object
 38  Services            10000 non-null  object
 39  Initial_days        10000 non-null  float64
 40  TotalCharge         10000 non-null  float64
 41  Additional_charges  10000 non-null  float64
 42  Item1               10000 non-null  int64
 43  Item2               10000 non-null  int64
 44  Item3               10000 non-null  int64
 45  Item4               10000 non-null  int64
 46  Item5               10000 non-null  int64
 47  Item6               10000 non-null  int64
 48  Item7               10000 non-null  int64
 49  Item8               10000 non-null  int64
dtypes: float64(7), int64(16), object(27)
memory usage: 3.8+ MB
```

In [4]:
```
#Selecting relevant columns for the analysis
```

```
columns = ["ReAdmis", "Overweight", "HighBlood", "Initial_days", "Doc_visits", "Initial_admin"]
```

In [5]:
```
#Retaining only the selected columns needed for the analysis
df = df[columns]
```

In [6]:
```
#Checking for duplicate values
print(df.duplicated())
```

```
0       False
1       False
2       False
3       False
4       False
        ...
9995    False
9996    False
9997    False
9998    False
9999    False
Length: 10000, dtype: bool
```

In [7]:
```
#Checking for missing values
print(df.isnull().sum())
```

```
ReAdmis          0
Overweight       0
HighBlood        0
Initial_days     0
Doc_visits       0
Initial_admin    0
dtype: int64
```

In [8]:
```
#Converting categorical variable Readmis to a numeric variable
df['ReAdmis'] = df['ReAdmis'].map({'Yes': 1, 'No': 0})
```

In [9]:
```
#Converting categorical variable Overweight to a numeric variable
df['Overweight'] = df['Overweight'].map({'Yes': 1, 'No': 0})
```

In [10]:
```
#Converting categorical variable HighBlood to a numeric variable
df['HighBlood'] = df['HighBlood'].map({'Yes': 1, 'No': 0})
```

In [11]:
```
#Converting categorical variable Initial_admin to a numeric variable
df['Initial_admin'] = df['Initial_admin'].map({'Emergency Admission':2, 'Elective Admission': 1,
```

In [12]:
```
df.info()
```

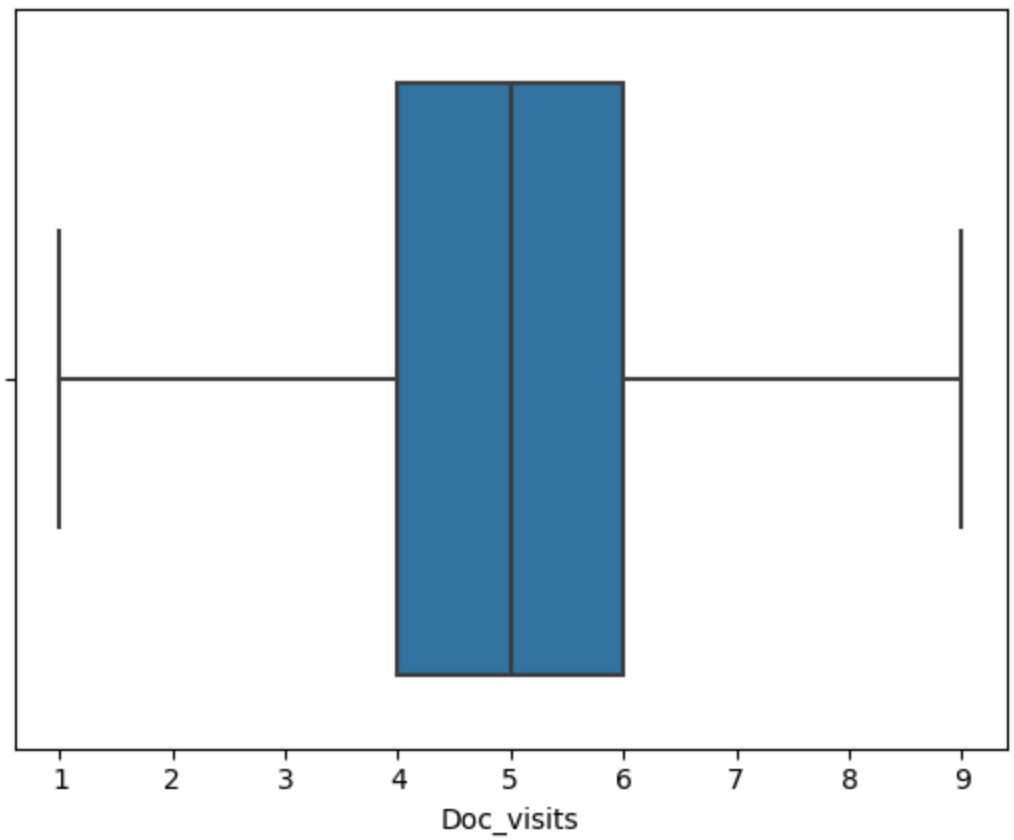
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   ReAdmis        10000 non-null  int64
 1   Overweight     10000 non-null  int64
 2   HighBlood      10000 non-null  int64
 3   Initial_days   10000 non-null  float64
 4   Doc_visits     10000 non-null  int64
 5   Initial_admin  10000 non-null  int64
dtypes: float64(1), int64(5)
memory usage: 468.9 KB
```

In [13]:
```
#List of numerical columns for checking outliers
numerical_columns = ['Overweight', 'HighBlood', 'Initial_days', 'Doc_visits', 'ReAdmis']

#Identify outliers based on Z-scores
z_scores = np.abs(zscore(df[numerical_columns]))
z_score_outliers = (z_scores > 3).sum(axis=0)
print("Outliers based on Z-score method:")
print(z_score_outliers)
```

```
Outliers based on Z-score method:
Overweight      0
HighBlood       0
Initial_days    0
Doc_visits      8
ReAdmis         0
dtype: int64
```

In [14]:
```
#Checking for outliers for Doc_visits
sns.boxplot(x = 'Doc_visits', data = df)
```

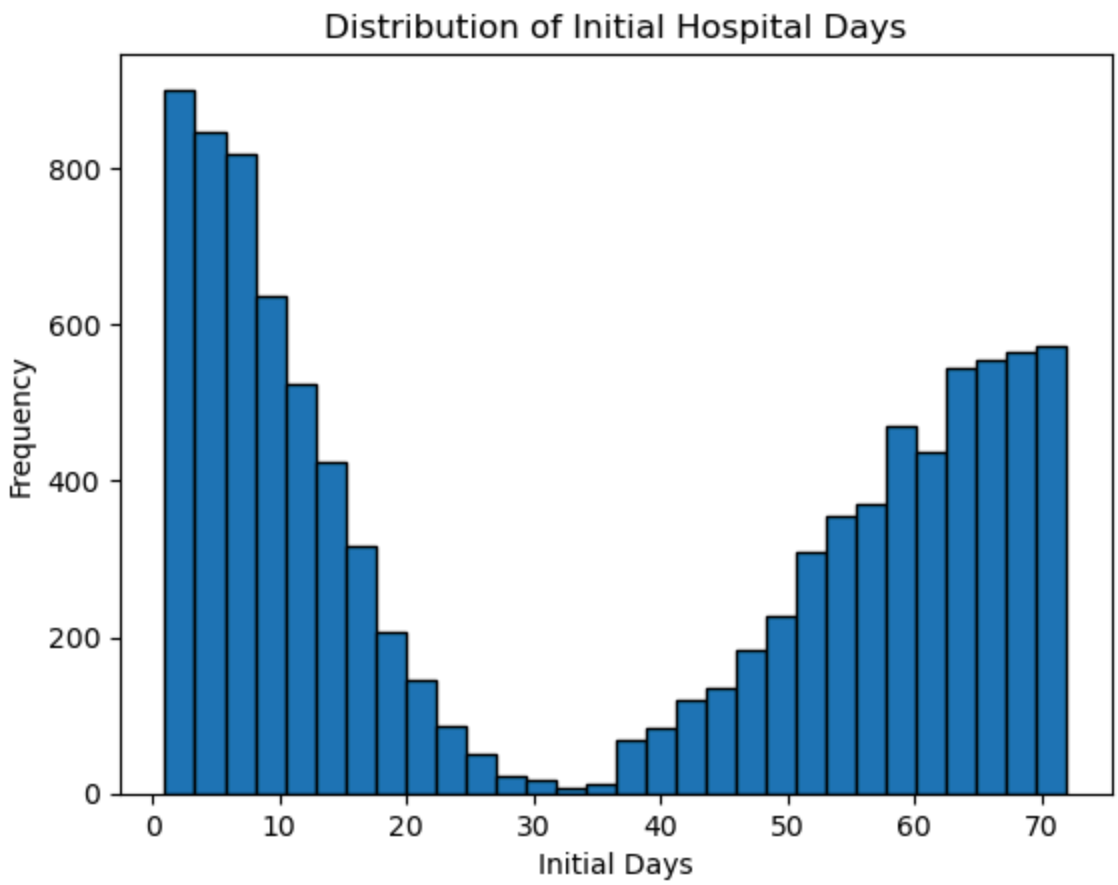
Out[14]:
```
<Axes: xlabel='Doc_visits'>
```



In [15]:
```
#Summary statistics for Initial days
print(df['Initial_days'].describe())
```

```
count    10000.000000
mean        34.455299
std         26.309341
min          1.001981
25%          7.896215
50%         35.836244
75%         61.161020
max         71.981490
Name: Initial_days, dtype: float64
```

In [16]:
```
# Continuous variable 1: Distribution of Initial_days
plt.hist(df['Initial_days'], bins=30, edgecolor='black')
plt.title('Distribution of Initial Hospital Days')
plt.xlabel('Initial Days')
plt.ylabel('Frequency')
plt.show()
```

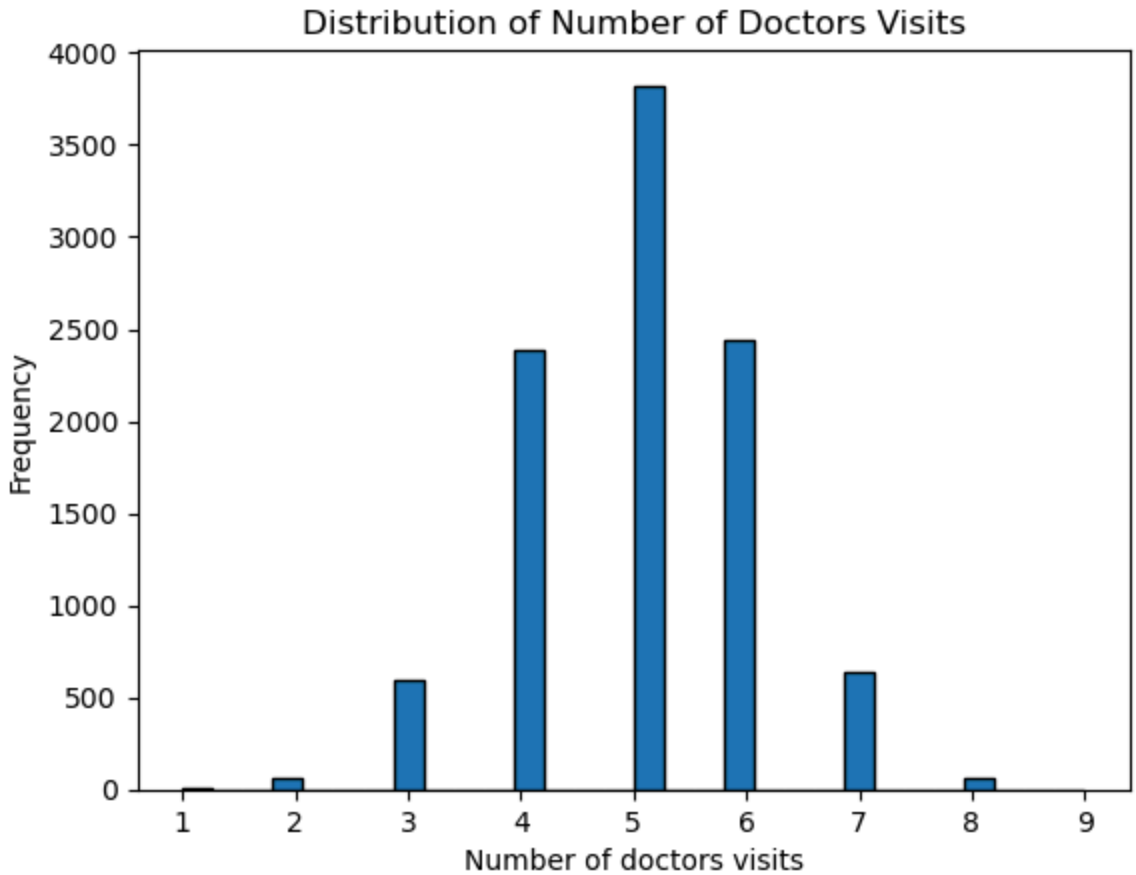


In [17]:
```
#Summary statistics for Doc_visits
print(df['Doc_visits'].describe())
```

```
count    10000.000000
mean         5.012200
std          1.045734
min          1.000000
25%          4.000000
50%          5.000000
75%          6.000000
max          9.000000
Name: Doc_visits, dtype: float64
```

In [18]:
```
# Continuous variable 1: Distribution of Doc_visits
plt.hist(df['Doc_visits'], bins=30, edgecolor='black')
```

```
plt.title('Distribution of Number of Doctors Visits')
plt.xlabel('Number of doctors visits')
plt.ylabel('Frequency')
plt.show()
```



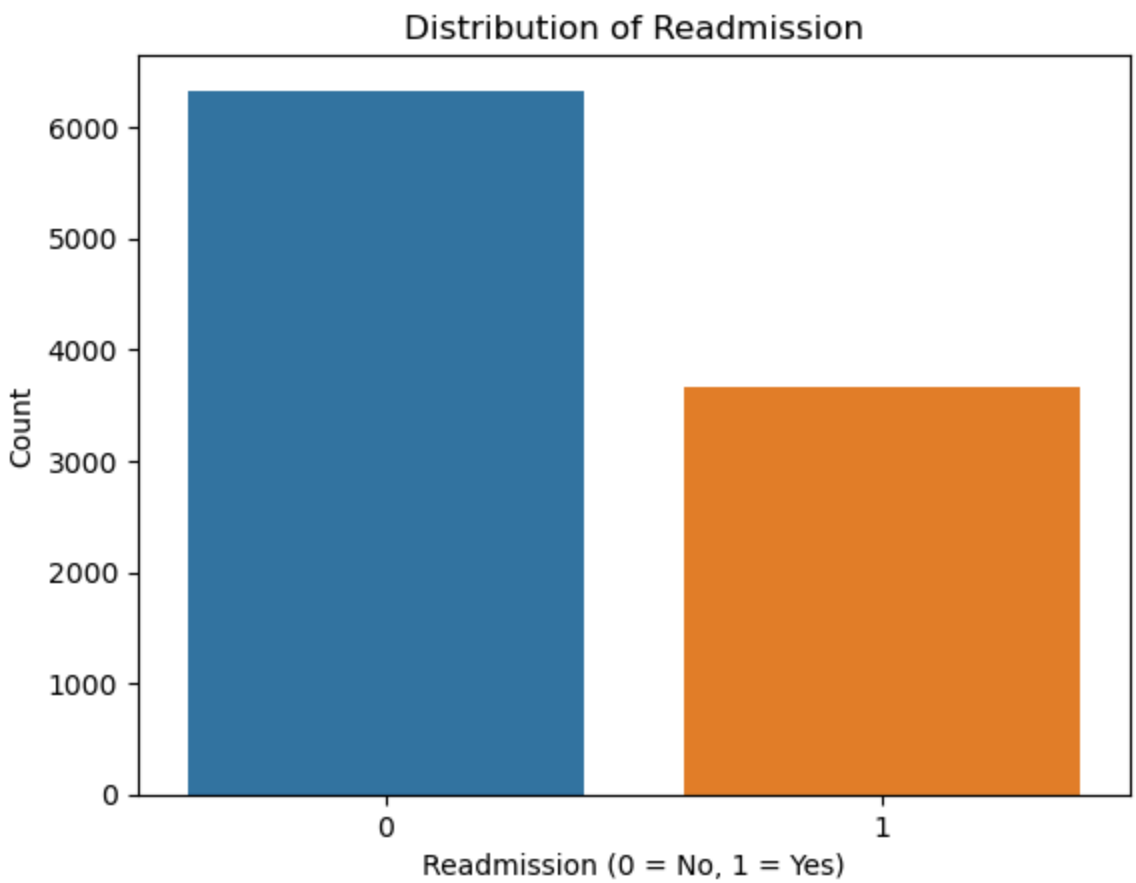
In [19]:
```
#Summary statistics for ReAdmis
print(df['ReAdmis'].describe())
```

```
count    10000.000000
mean         0.366900
std          0.481983
min          0.000000
25%          0.000000
50%          0.000000
75%          1.000000
max          1.000000
Name: ReAdmis, dtype: float64
```

In [20]:
```
#Value count for Readmission
print(df['ReAdmis'].value_counts())
#Univariate Statistics for ReAdmis variable
readmission = df['ReAdmis'].value_counts(normalize = True).mul(100).sort_values(ascending = Fals
print('Relative frequency for Readmission:', readmission)
```

```
ReAdmis
0    6331
1    3669
Name: count, dtype: int64
Relative frequency for Readmission: ReAdmis
0    63.31
1    36.69
Name: proportion, dtype: float64
```

In [21]:
```
#Univariate statistics for Readmission
sns.countplot(data=df, x='ReAdmis')
plt.title('Distribution of Readmission')
plt.xlabel('Readmission (0 = No, 1 = Yes)')
plt.ylabel('Count')
plt.show()
```



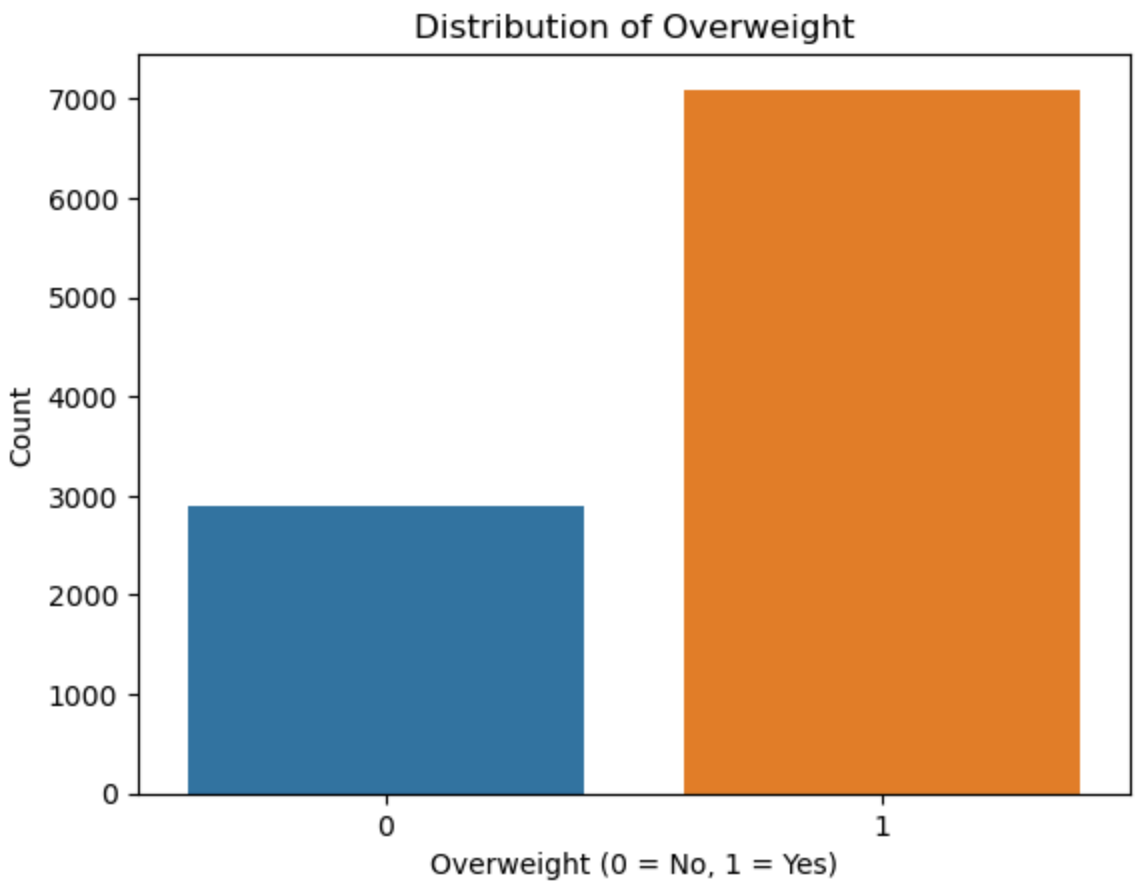
In [22]:
```
#Summary statistics for Overweight
print(df['Overweight'].describe())
```

```
count    10000.000000
mean         0.709400
std          0.454062
min          0.000000
25%          0.000000
50%          1.000000
75%          1.000000
max          1.000000
Name: Overweight, dtype: float64
```

In [23]:
```
#Value count for Overweight
print(df['Overweight'].value_counts())
#Univariate Statistics for Overweight variable
overweight = df['Overweight'].value_counts(normalize = True).mul(100).sort_values(ascending = Fa
print('Relative frequency for Overweight:', overweight)
```

```
Overweight
1    7094
0    2906
Name: count, dtype: int64
Relative frequency for Overweight: Overweight
1    70.94
0    29.06
Name: proportion, dtype: float64
```

In [24]:
```
#Univariate statistics for Overweight
sns.countplot(data=df, x='Overweight')
plt.title('Distribution of Overweight')
plt.xlabel('Overweight (0 = No, 1 = Yes)')
plt.ylabel('Count')
plt.show()
```



In [25]:
```
#Summary statistics for HighBlood
print(df['HighBlood'].describe())
```

```
count    10000.000000
mean         0.409000
std          0.491674
min          0.000000
25%          0.000000
50%          0.000000
75%          1.000000
max          1.000000
Name: HighBlood, dtype: float64
```

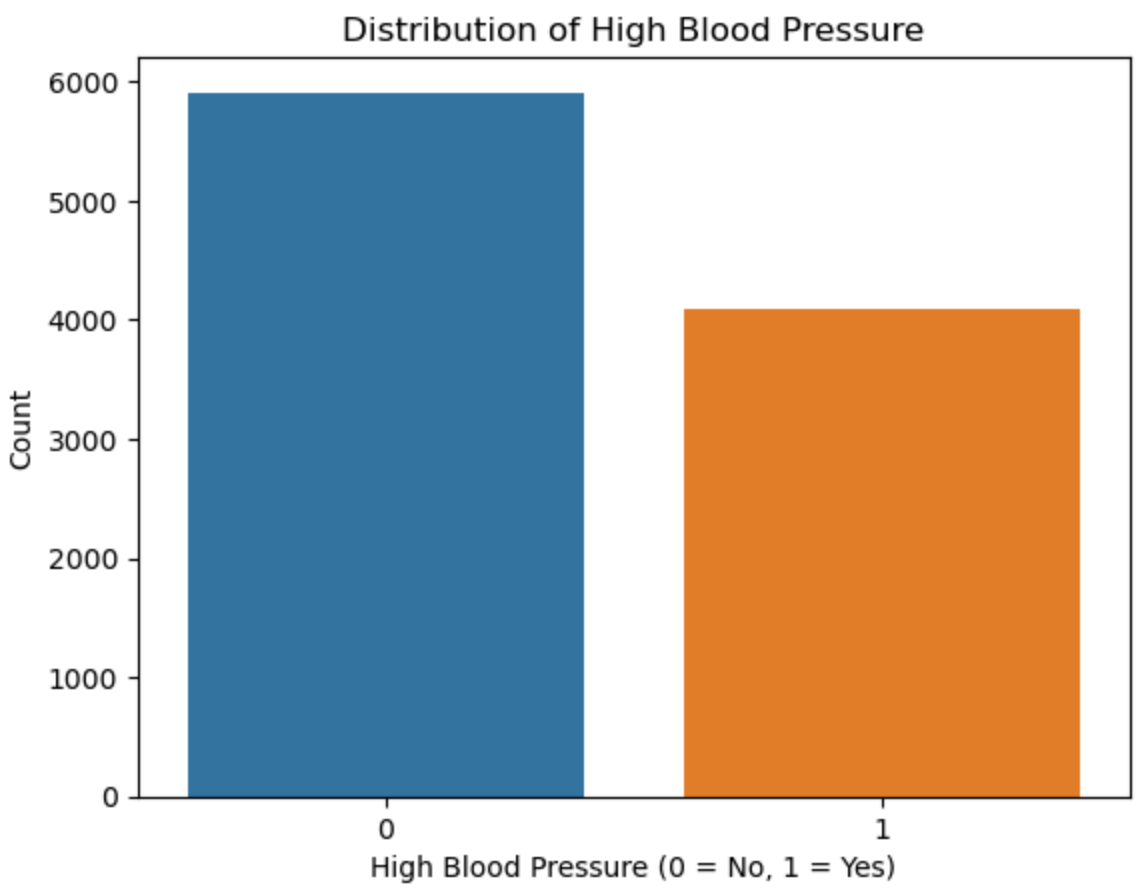
In [26]:
```
#Value count for HighBlood
print(df['HighBlood'].value_counts())
#Univariate Statistics for HighBlood variable
```

```
highblood = df['HighBlood'].value_counts(normalize = True).mul(100).sort_values(ascending = Fals
print('Relative frequency for High Blood Pressure:', highblood)
```

```
HighBlood
0    5910
1    4090
Name: count, dtype: int64
Relative frequency for High Blood Pressure: HighBlood
0    59.1
1    40.9
Name: proportion, dtype: float64
```

In [27]:

```
#Univariate statistics for HighBlood
sns.countplot(data=df, x='HighBlood')
plt.title('Distribution of High Blood Pressure')
plt.xlabel('High Blood Pressure (0 = No, 1 = Yes)')
plt.ylabel('Count')
plt.show()
```



In [28]:

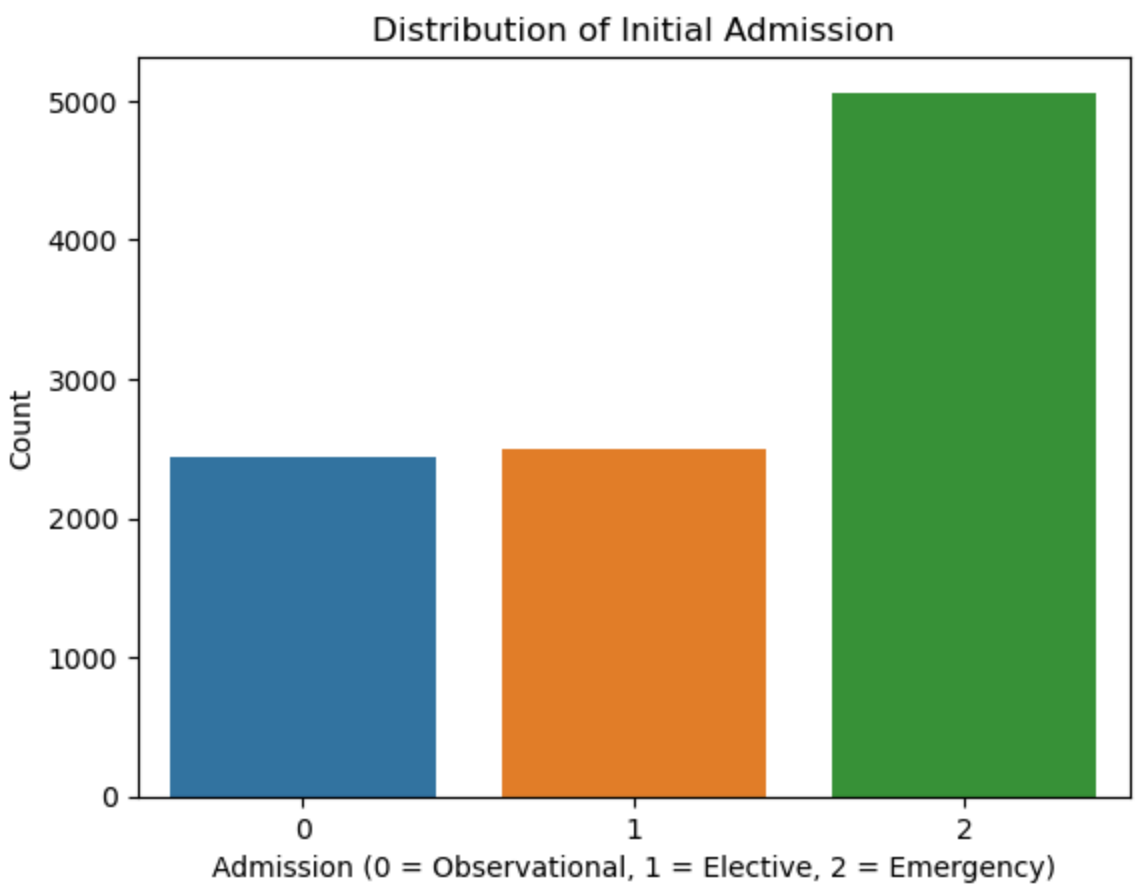
```
#Summary statistics for Initial admission
print(df['Initial_admin'].describe())
```

```
count    10000.000000
mean         1.262400
std          0.825115
min          0.000000
25%          1.000000
50%          2.000000
75%          2.000000
max          2.000000
Name: Initial_admin, dtype: float64
```

In [29]:
```
#Value count for Initial Admission
print(df['Initial_admin'].value_counts())
#Univariate Statistics for Initial Admission variable
admission = df['Initial_admin'].value_counts(normalize = True).mul(100).sort_values(ascending = 
print('Relative frequency for Initial Admission:', admission)
```

```
Initial_admin
2    5060
1    2504
0    2436
Name: count, dtype: int64
Relative frequency for Initial Admission: Initial_admin
2    50.60
1    25.04
0    24.36
Name: proportion, dtype: float64
```

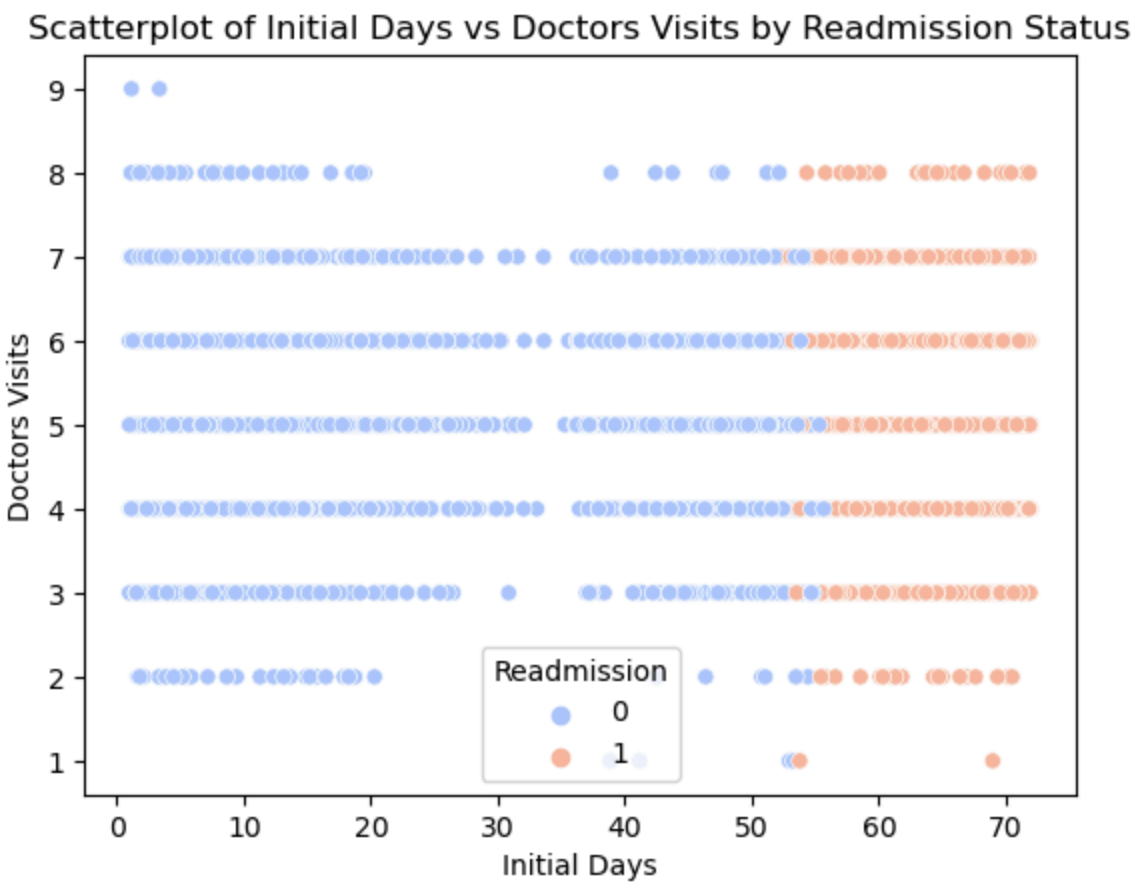
In [30]:
```
#Univariate statistics for Initial Admission
sns.countplot(data=df, x='Initial_admin')
plt.title('Distribution of Initial Admission')
plt.xlabel('Admission (0 = Observational, 1 = Elective, 2 = Emergency)')
plt.ylabel('Count')
plt.show()
```



In [31]:
```
#Bivariate analysis for Initial_days and Doc_visits variables
pearson, _ = stats.pearsonr(df.Initial_days, df.Doc_visits)
print("Pearson correlation coefficient:")
print(pearson)
```

```
Pearson correlation coefficient:
-0.006754303231524527
```

In [32]:
```
#Bivariate statistics for Initial Days vs Doctor Visits by Readmission Status
sns.scatterplot(data=df, x='Initial_days', y='Doc_visits', hue='ReAdmis', palette='coolwarm')
plt.title('Scatterplot of Initial Days vs Doctors Visits by Readmission Status')
plt.xlabel('Initial Days')
plt.ylabel('Doctors Visits')
plt.legend(title='Readmission')
plt.show()
```

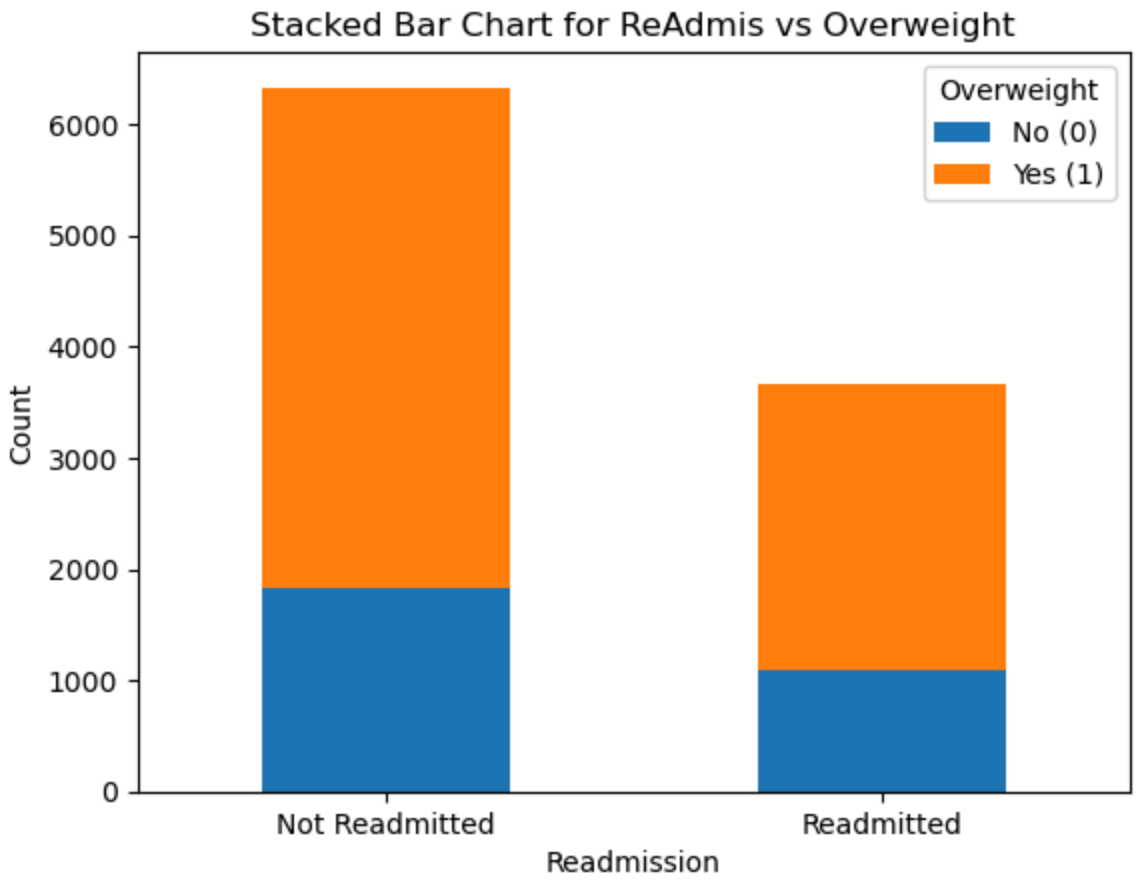


In [33]:
```
#Bivariate analysis for Readmission and Overweight
#Contingency table
contingency_table_r_o = pd.crosstab(df['ReAdmis'], df['Overweight'])

#Perform the chi-square test of independence
chi2, p, dof, expected = stats.chi2_contingency(contingency_table_r_o)

#Calculate Cramér's V
#Total number of observations
n = contingency_table_r_o.sum().sum()
#Minimum dimension of the contingency table minus 1
min_dim = min(contingency_table_r_o.shape) - 1
cramers_v = np.sqrt(chi2 / (n * min_dim))
print('Chi squared test:', chi2)
print('P-value for chi squared test:', p)
print('Cramers_v value:',cramers_v)
```

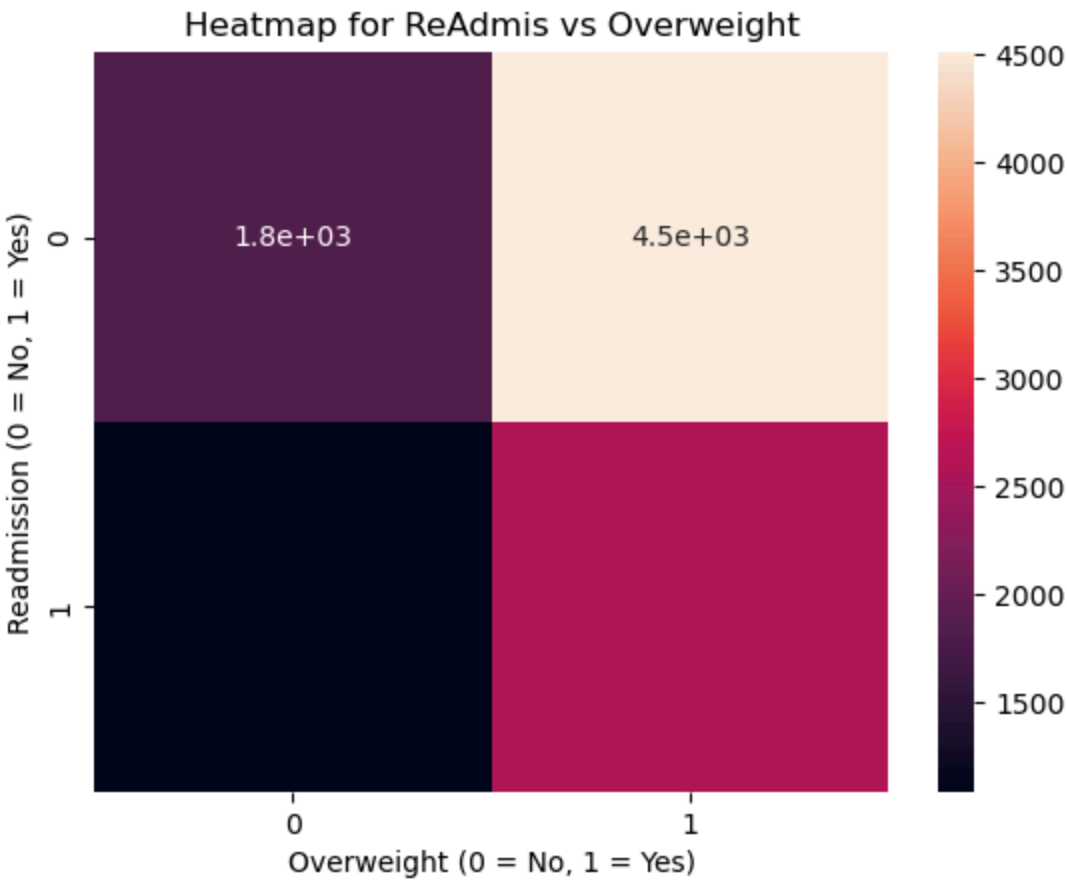
```
Chi squared test: 0.6984802059617877
P-value for chi squared test: 0.4032948387365497
Cramers_v value: 0.0083575128235725
```

In [34]:
```
#Bivariate Stacked Bar Chart for Readmis and Overweight
contingency_table_r_o.plot(kind='bar', stacked=True)
plt.title('Stacked Bar Chart for ReAdmis vs Overweight')
```

```
plt.xlabel('Readmission')
plt.ylabel('Count')
plt.xticks([0, 1], ['Not Readmitted', 'Readmitted'], rotation=0)
plt.legend(title='Overweight', labels=['No (0)', 'Yes (1)'])
plt.show()
```



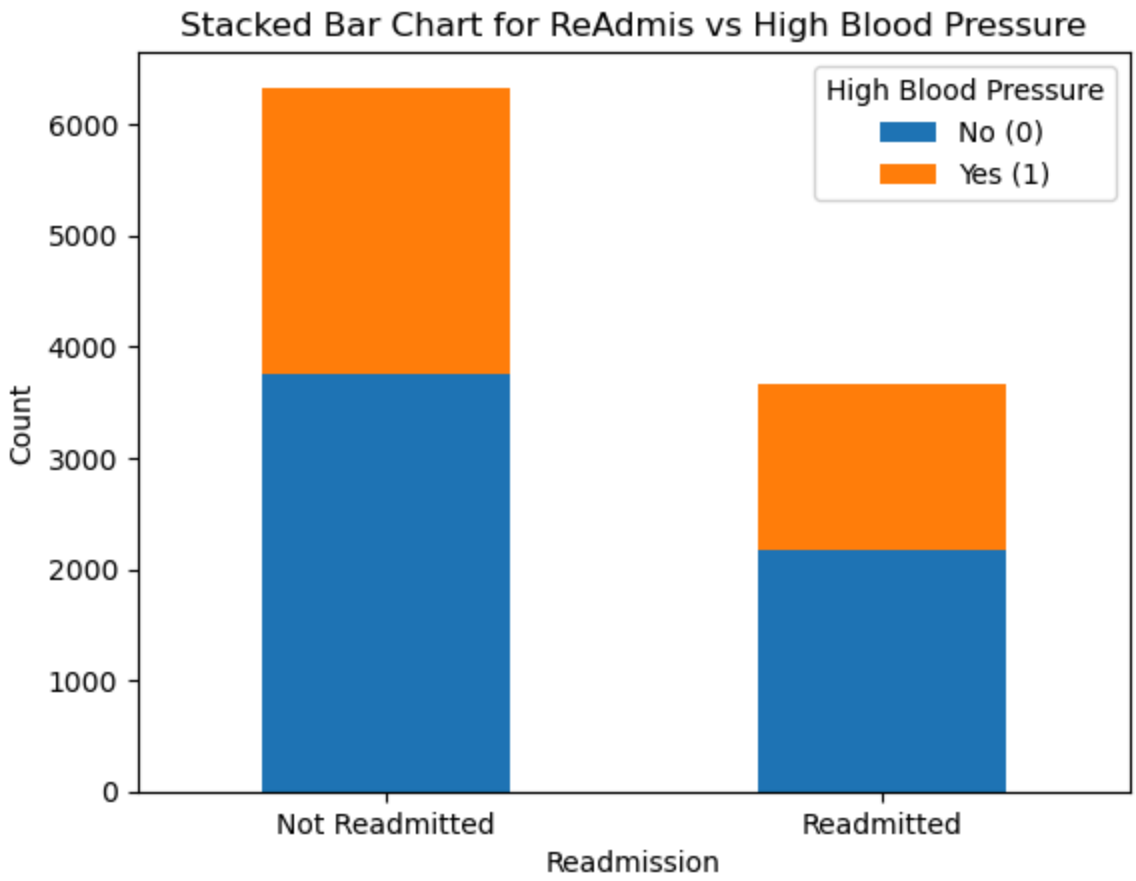
In [35]:
```
#Bivariate Heatmap for Readmis and Overweight
sns.heatmap(contingency_table_r_o, annot = True)
plt.title('Heatmap for ReAdmis vs Overweight')
plt.xlabel('Overweight (0 = No, 1 = Yes)')
plt.ylabel('Readmission (0 = No, 1 = Yes)')
plt.show()
```



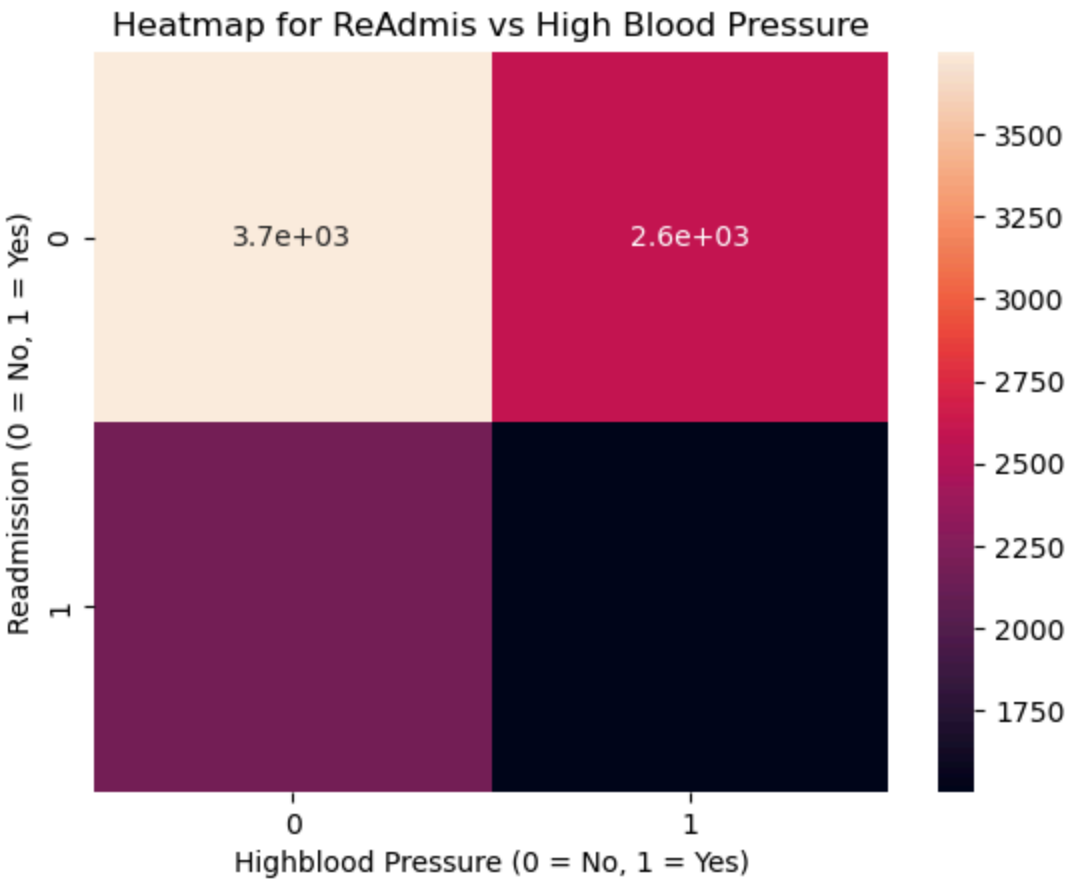
In [36]:
```
#Bivariate analysis for Readmission and Highblood
#Contingency table
contingency_table_r_h = pd.crosstab(df['ReAdmis'], df['HighBlood'])

#Perform the chi-square test of independence
chi2, p, dof, expected = stats.chi2_contingency(contingency_table_r_h)

#Calculate Cramér's V
#Total number of observations
n = contingency_table_r_h.sum().sum()
#Minimum dimension of the contingency table minus 1
min_dim = min(contingency_table_r_h.shape) - 1
cramers_v = np.sqrt(chi2 / (n * min_dim))
print('Chi squared test:', chi2)
print('P-value for chi squared test:', p)
print('Cramers_v value:',cramers_v)
```

```
Chi squared test: 0.04239657973011679
P-value for chi squared test: 0.8368656684578771
Cramers_v value: 0.002059042975027884
```

In [37]:
```
#Bivariate Stacked Bar Chart for Readmis and HighBlood
contingency_table_r_h.plot(kind='bar', stacked=True)
plt.title('Stacked Bar Chart for ReAdmis vs High Blood Pressure')
plt.xlabel('Readmission')
plt.ylabel('Count')
plt.xticks([0, 1], ['Not Readmitted', 'Readmitted'], rotation=0)
plt.legend(title='High Blood Pressure', labels=['No (0)', 'Yes (1)'])
plt.show()
```



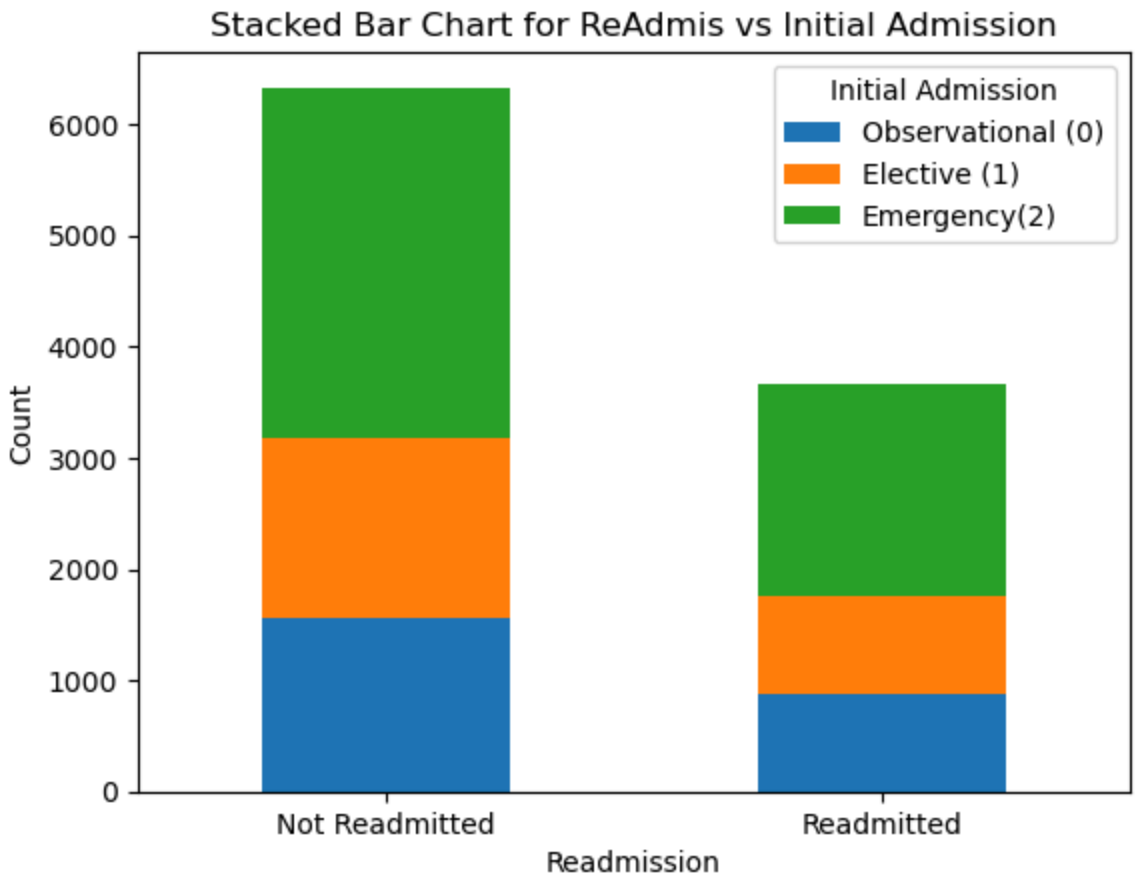
In [38]:
```
#Bivariate Heatmap for Readmis and HighBlood
sns.heatmap(contingency_table_r_h, annot = True)
plt.title('Heatmap for ReAdmis vs High Blood Pressure')
plt.xlabel('Highblood Pressure (0 = No, 1 = Yes)')
plt.ylabel('Readmission (0 = No, 1 = Yes)')
plt.show()
```



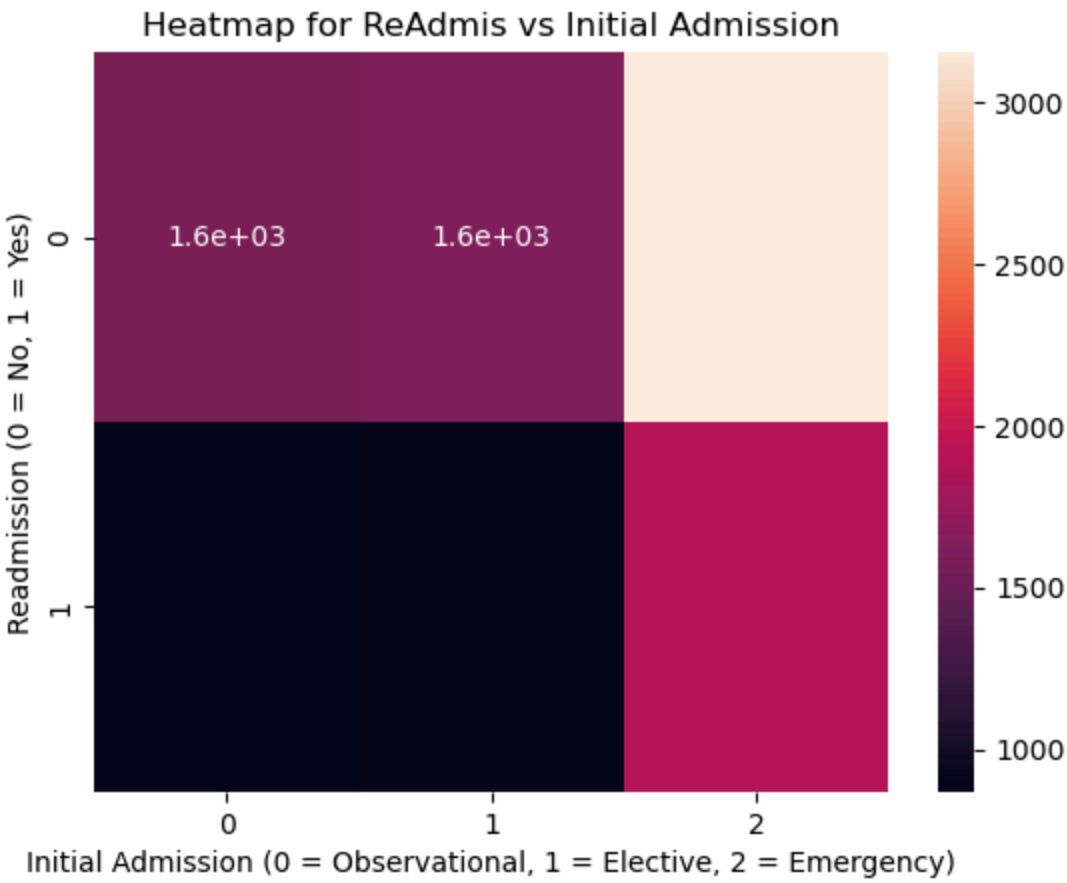
In [39]:
```
#Bivariate analysis for Readmission and Initial_admin
#Contingency table
contingency_table_r_i = pd.crosstab(df['ReAdmis'], df['Initial_admin'])

#Perform the chi-square test of independence
chi2, p, dof, expected = stats.chi2_contingency(contingency_table_r_i)

#Calculate Cramér's V
#Total number of observations
n = contingency_table_r_i.sum().sum()
#Minimum dimension of the contingency table minus 1
min_dim = min(contingency_table_r_i.shape) - 1
cramers_v = np.sqrt(chi2 / (n * min_dim))
print('Chi squared test:', chi2)
print('P-value for chi squared test:', p)
print('Cramers_v value:',cramers_v)
```

```
Chi squared test: 3.8899679903336484
P-value for chi squared test: 0.14298951184306222
Cramers_v value: 0.01972300177542366
```

In [40]:
```
#Bivariate Stacked Bar Chart for Readmis and Initial_admin
contingency_table_r_i.plot(kind='bar', stacked=True)
plt.title('Stacked Bar Chart for ReAdmis vs Initial Admission')
plt.xlabel('Readmission')
plt.ylabel('Count')
plt.xticks([0, 1], ['Not Readmitted', 'Readmitted'], rotation=0)
plt.legend(title='Initial Admission', labels=['Observational (0)', 'Elective (1)','Emergency(2)'
plt.show()
```



In [41]:
```
#Bivariate Heatmap for Readmis and Initial_admin
sns.heatmap(contingency_table_r_i, annot = True)
plt.title('Heatmap for ReAdmis vs Initial Admission')
plt.xlabel('Initial Admission (0 = Observational, 1 = Elective, 2 = Emergency)')
plt.ylabel('Readmission (0 = No, 1 = Yes)')
plt.show()
```



In [42]:

```
#Initial Logistic Regression Model
#Define dependent and independent variables
X = df[['Overweight', 'HighBlood', 'Initial_days', 'Doc_visits', 'Initial_admin']]
y = df['ReAdmis']

#Add a constant to the model
X = sm.add_constant(X)

#Split the data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

#Fit the logistic regression model
logit_model = sm.Logit(y_train, X_train)
initial_model = logit_model.fit()

#Model summary
print(initial_model.summary())
```

```
Optimization terminated successfully.
         Current function value: 0.047668
         Iterations 13
                           Logit Regression Results
==============================================================================
Dep. Variable:                ReAdmis   No. Observations:                 8000
Model:                          Logit   Df Residuals:                     7994
Method:                           MLE   Df Model:                            5
Date:                Mon, 11 May 2026   Pseudo R-squ.:                  0.9277
Time:                        06:09:39   Log-Likelihood:                -381.35
converged:                       True   LL-Null:                       -5271.6
Covariance Type:            nonrobust   LLR p-value:                     0.000
===================================================================================
                     coef    std err          z      P>|z|      [0.025      0.975]
-----------------------------------------------------------------------------------
const            -56.3857      2.980    -18.921      0.000     -62.227     -50.545
Overweight        -0.0848      0.202     -0.421      0.674      -0.480       0.310
HighBlood          0.6631      0.194      3.413      0.001       0.282       1.044
Initial_days       1.0193      0.053     19.130      0.000       0.915       1.124
Doc_visits        -0.0214      0.085     -0.251      0.802      -0.188       0.146
Initial_admin      0.7073      0.118      5.979      0.000       0.475       0.939
===================================================================================

Possibly complete quasi-separation: A fraction 0.74 of observations can be
perfectly predicted. This might indicate that there is complete
quasi-separation. In this case some parameters will not be identified.
```

In [43]:

```python
#Correlation Matrix
X_corr = df[['Overweight', 'HighBlood', 'Initial_days', 'Doc_visits', 'Initial_admin']]
corr_matrix = pd.DataFrame(X_corr).corr()
print(corr_matrix)
```

```
               Overweight  HighBlood  Initial_days  Doc_visits  Initial_admin
Overweight       1.000000   0.026231     -0.011077    0.011890      -0.011069
HighBlood        0.026231   1.000000     -0.006333    0.008967      -0.003998
Initial_days    -0.011077  -0.006333      1.000000   -0.006754      -0.008482
Doc_visits       0.011890   0.008967     -0.006754    1.000000      -0.005913
Initial_admin   -0.011069  -0.003998     -0.008482   -0.005913       1.000000
```

In [44]:

```python
#Variance Inflation Factor (VIF)
#Define the independent variables
X_vif = df[['Overweight', 'HighBlood', 'Initial_days', 'Doc_visits', 'Initial_admin']]

#Add a constant to the independent variables matrix (for intercept in the model)
X_vif = sm.add_constant(X_vif)

#Calculate VIF for each feature
vif_data = pd.DataFrame()
vif_data['Variable'] = X_vif.columns
vif_data['VIF'] = [variance_inflation_factor(X_vif.values, i) for i in range(X_vif.shape[1])]

#Display the VIF for each variable
print(vif_data)
```

```
        Variable        VIF
0          const  31.189606
1     Overweight   1.001063
2      HighBlood   1.000813
3   Initial_days   1.000278
4     Doc_visits   1.000293
5  Initial_admin   1.000244
```

In [45]:

```
#Reduced Logistic Regression Model
#Define X values for the reduced model
X_reduced = df[['HighBlood', 'Initial_days','Initial_admin']]

#Add a constant to the model
X_reduced = sm.add_constant(X_reduced)

#Split the data into training and testing sets (80% train, 20% test)
X_train_reduced, X_test_reduced, y_train, y_test = train_test_split(X_reduced, y, test_size=0.2,

#Fit the logistic regression model
logit_model_reduced = sm.Logit(y_train, X_train_reduced)
reduced_model= logit_model_reduced.fit()

#Model summary
print(reduced_model.summary())
```

```
Optimization terminated successfully.
         Current function value: 0.047683
         Iterations 13
                           Logit Regression Results
==============================================================================
Dep. Variable:                ReAdmis   No. Observations:                 8000
Model:                          Logit   Df Residuals:                     7996
Method:                           MLE   Df Model:                            3
Date:                Mon, 11 May 2026   Pseudo R-squ.:                  0.9276
Time:                        06:09:40   Log-Likelihood:                -381.46
converged:                       True   LL-Null:                       -5271.6
Covariance Type:            nonrobust   LLR p-value:                     0.000
=================================================================================
                    coef    std err          z      P>|z|      [0.025      0.975]
---------------------------------------------------------------------------------
const           -56.4735      2.958    -19.093      0.000     -62.271     -50.676
HighBlood         0.6560      0.194      3.388      0.001       0.277       1.035
Initial_days      1.0179      0.053     19.162      0.000       0.914       1.122
Initial_admin     0.7072      0.118      5.982      0.000       0.475       0.939
=================================================================================

Possibly complete quasi-separation: A fraction 0.74 of observations can be
perfectly predicted. This might indicate that there is complete
quasi-separation. In this case some parameters will not be identified.
```

In [46]:

```
#Making predictions using the reduced model
y_pred = reduced_model.predict(X_test_reduced)
#Turning the predictions into binary predictions 0 for No and 1 for Yes
y_pred_class = (y_pred >= 0.5).astype(int)
#Print the binary prediction probability
print(y_pred_class)
```

```
6252    0
4684    0
1731    0
4742    0
4521    0
       ..
6412    1
8285    0
7853    1
1095    0
6929    0
Length: 2000, dtype: int32
```

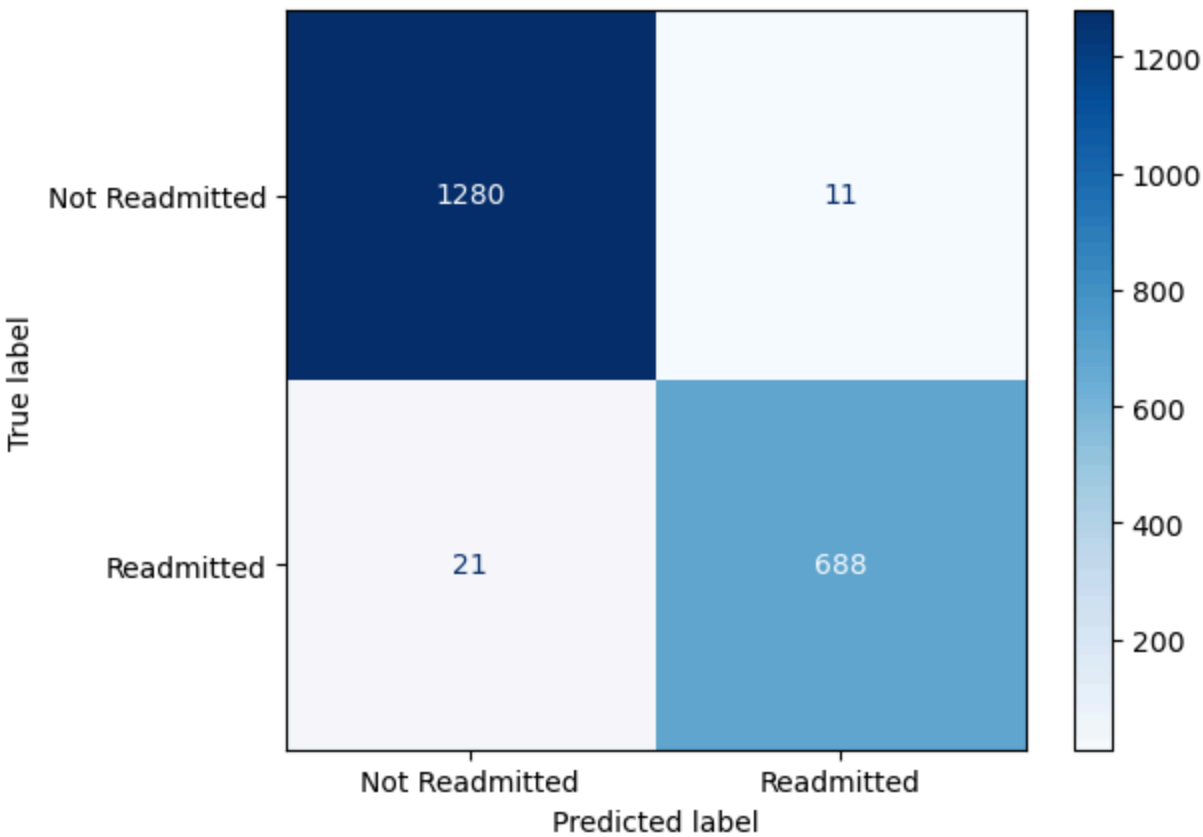
In [47]:
```
#Calculating Accuracy
reduced_model_accuracy = accuracy_score(y_test, y_pred_class)
print(f"Accuracy: {reduced_model_accuracy:.4f}")
```

```
Accuracy: 0.9840
```

In [48]:
```
#Confusion Matrix
conf_matrix = confusion_matrix(y_test, y_pred_class)
print(conf_matrix)
```

```
[[1280   11]
 [  21  688]]
```

In [49]:
```
#Create the ConfusionMatrixDisplay object
cm_plot = ConfusionMatrixDisplay(confusion_matrix=conf_matrix, display_labels=["Not Readmitted",
cm_plot.plot(cmap='Blues')
plt.show()
```



In [50]:
```
coefficients = reduced_model.params

#Build the regression equation
```

```
terms = [f"{coef:.2f}*{var}" if var != "const" else f"{coef:.2f}" for var, coef in coefficients.
equation = " + ".join(terms)

#Print the regression equation
print("Logistic Regression Equation:")
print(f"Readmission = {equation}")
```

```
Logistic Regression Equation:
Readmission = -56.47 + 0.66*HighBlood + 1.02*Initial_days + 0.71*Initial_admin
```

In [51]:

```
#Save cleaned dataset
new_data_path = 'output/new_medical_data_logistic_D208_II.csv'
df.to_csv(new_data_path, index=False)
```

In [ ]: